

Smart Waste Detection

Garbage Classification Using Classical Machine Learning and Transfer Learning

CSS 324: Introduction to Machine Learning

Final Project Report

Project Type: Image Classification
Final Model: EfficientNetB0 Transfer Learning
Dataset: Garbage Classification V2
Final Accuracy: 92.68%

Team Members

Zhakupbayev Aisultan
Kuttybaev Islam
Shaimurzyn Abylay
Anuar Diana
Anel Kakimbek

SDU University

Spring 2026

May 12, 2026

Contents

1	Topic and Motivation	2
2	Abstract	2
3	Related Work	2
4	Exploratory Data Analysis	3
4.1	Dataset Description	3
4.2	Dataset Size and Class Distribution	4
4.3	Data Preparation	5
4.4	Expected Modeling Challenges	6
4.5	EDA Conclusion	6
5	Classical Machine Learning Models	6
5.1	Preprocessing	7
5.2	Models	7
5.3	Classical Model Results	7
6	Transfer Learning with EfficientNetB0	8
6.1	Reason for Choosing EfficientNetB0	8
6.2	Data Loading	8
6.3	Data Augmentation	9
7	EfficientNetB0 Architecture	10
7.1	Parameter Summary	10
8	Training	11
8.1	Phase 1: Feature Extraction	11
8.2	Training Accuracy Curve	12
8.3	Training Loss Curve	12
9	Final Evaluation	13
10	Per-Class Metrics	13
10.1	F1-score by Class	14
10.2	Precision vs Recall	14
11	Confusion Matrix and Error Analysis	14
11.1	Absolute Confusion Matrix	15
11.2	Normalized Confusion Matrix	15
12	Final Model Comparison	15
13	Final Model and Demo Description	16
14	Limitations	16
15	Future Work	17
16	Analysis and Conclusions	17

1. Topic and Motivation

This project was completed for **CSS 324: Introduction to Machine Learning**. The final project required a real-world machine learning problem, dataset preparation, exploratory data analysis, at least three trained models, model comparison, error analysis, and an interactive prediction demonstration.

The selected topic is **Smart Waste Detection**. The task is image classification: given an image of a waste item, the model predicts its category.

This problem is important because correct waste sorting improves recycling efficiency and reduces environmental damage. Machine learning can help automate sorting in smart bins, recycling centers, and waste management systems.

2. Abstract

This project develops a garbage classification system using classical machine learning and transfer learning. The dataset contains **12,259 images** from **10 waste categories**. The data was split into training, validation, and test sets using an 80% / 10% / 10% ratio.

Four models were evaluated:

- Logistic Regression,
- Support Vector Machine with RBF kernel,
- Random Forest,
- EfficientNetB0 transfer learning model.

The best classical model was Random Forest with **58.21%** test accuracy. The final EfficientNetB0 transfer learning model achieved **92.68%** test accuracy, clearly outperforming the classical baselines.

3. Related Work

Several studies have already shown that deep learning is effective for garbage classification.

White et al. proposed *WasteNet*, a waste classification model designed for smart bins and edge devices such as Jetson Nano [1]. Their model classified six categories: paper, cardboard, glass, metal, plastic, and other. WasteNet focused strongly on **edge deployment** and used a hybrid transfer learning strategy with gradual unfreezing. Their reported accuracy was **97%**. In comparison, our project focuses on a larger 10-class dataset and compares both classical ML baselines and EfficientNetB0 transfer learning.

Al-Mamun et al. developed a CNN-based garbage classification system using six classes from a Kaggle dataset: cardboard, glass, metal, paper, plastic, and trash [2]. Their work used a custom CNN architecture with convolution, pooling, batch normalization, dropout, and dense layers. They reported high training accuracy of **98.45%**, but their evaluation accuracy was lower, around **85.53%**. The main difference from our report is that our final model uses **transfer learning** with EfficientNetB0 instead of training a CNN from scratch.

These studies support the choice of CNN-based and transfer-learning methods for image-based waste classification. They also show that garbage classification performance depends strongly on

dataset size, class balance, preprocessing, and whether pretrained image features are used.

Work	Model	Dataset / Classes	Main Result
WasteNet [1]	DenseNet-based transfer learning	TrashNet, 6 classes	97% accuracy
Al-Mamun et al. [2]	Custom CNN	Kaggle garbage dataset, 6 classes	98.45% training accuracy, 85.53% evaluation accuracy
Our Project	EfficientNetB0 transfer learning	Garbage Classification V2, 10 classes	92.68% test accuracy

4. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the dataset structure, class balance, data preparation process, and expected modeling challenges before training the models.

4.1 Dataset Description

The dataset was downloaded from Kaggle:

```
!kaggle datasets download -d sumn2u/garbage-classification-v2
```

The standardized image folder was extracted:

```
import zipfile, os

zip_path = "garbage-classification-v2.zip"
extract_path = "garbage_dataset"
target_folder = "standardized_256/"

with zipfile.ZipFile(zip_path, "r") as z:
    selected = [f for f in z.namelist() if f.startswith(target_folder)]
    for f in selected:
        z.extract(f, extract_path)
```

After extraction, the dataset contained **12,259 images** from **10 classes**.

Class	Number of Images
battery	756
biological	699
cardboard	1411
clothes	1892
glass	1736
metal	930
paper	1336
plastic	1597
shoes	1449
trash	453
Total	12259

4.2 Dataset Size and Class Distribution

The dataset size is large enough for transfer learning, but the class distribution is not balanced. The largest class is *clothes* with 1,892 images, while the smallest class is *trash* with only 453 images.

The average number of images per class is approximately:

$$\frac{12259}{10} \approx 1225$$

The imbalance ratio between the largest and smallest classes is:

$$\frac{1892}{453} \approx 4.18$$

This means the largest class contains more than four times as many images as the smallest class. This is important because the model may learn majority classes better than minority classes.

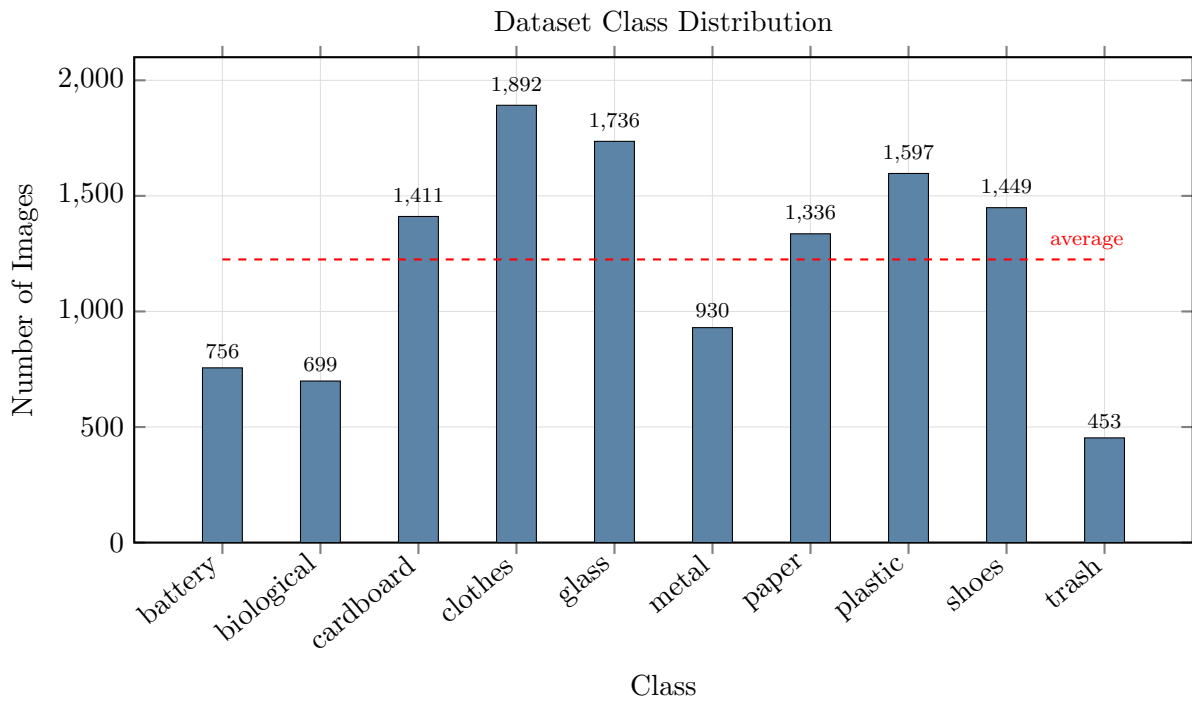


Figure 1: Class distribution of the dataset. The dashed red line shows the average number of images per class.

4.3 Data Preparation

The dataset was split into training, validation, and test sets using an 80% / 10% / 10% ratio.

```
from sklearn.model_selection import train_test_split

train_imgs, temp = train_test_split(
    images,
    test_size=0.2,
    random_state=42
)

val_imgs, test_imgs = train_test_split(
    temp,
    test_size=0.5,
    random_state=42
)
```

Split	Number of Images
Train	9802
Validation	1227
Test	1230

The split preserved class proportions across training, validation, and test sets. This is important because evaluation results should represent the same class distribution as the original dataset.

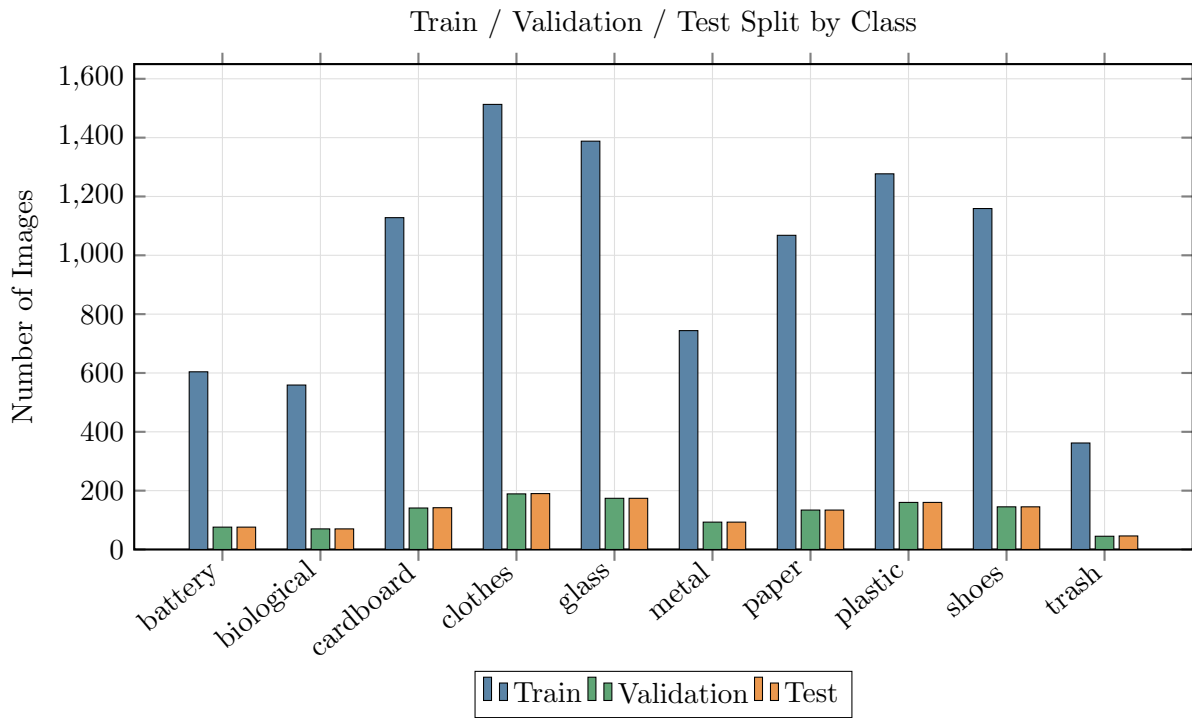


Figure 2: Train, validation, and test distribution by class.

4.4 Expected Modeling Challenges

Based on the dataset structure, several modeling challenges were expected before training:

1. **Class imbalance:** minority classes such as *trash*, *biological*, and *battery* have fewer samples.
2. **Similar visual appearance:** some classes look similar, such as glass vs plastic, paper vs cardboard, and metal vs trash.
3. **Loss of spatial information in classical ML:** flattening images for classical models removes spatial structure.
4. **Real-world variation:** images may have different lighting, backgrounds, object positions, and camera angles.

These challenges suggested that classical models may have limited performance and that transfer learning would be more suitable.

4.5 EDA Conclusion

The EDA showed that the dataset is useful for a real-world image classification task, but it is not simple. The dataset is moderately imbalanced, contains visually similar waste categories, and includes realistic image variation.

The class distribution and split graphs confirm that the data was prepared correctly and that all classes are represented in training, validation, and testing.

Overall, the EDA supports the decision to use EfficientNetB0 transfer learning, because pretrained convolutional features are better suited for image classification than flattened pixel vectors.

5. Classical Machine Learning Models

5.1 Preprocessing

For classical machine learning, images were resized to 64×64 and flattened.

$$64 \times 64 \times 3 = 12288$$

Each image became a vector with 12,288 numerical features.

```
IMG_SIZE_ML = 64

X_train_flat = (X_train / 255.0).reshape(len(X_train), -1)
X_val_flat = (X_val / 255.0).reshape(len(X_val), -1)
X_test_flat = (X_test / 255.0).reshape(len(X_test), -1)
```

5.2 Models

Three classical models were trained:

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

log_reg = LogisticRegression(max_iter=1000, n_jobs=-1)
svm_model = SVC(kernel="rbf")
rf_model = RandomForestClassifier(
    n_estimators=100,
    n_jobs=-1,
    random_state=42
)
```

5.3 Classical Model Results

Model	Test Accuracy
Logistic Regression	34.96%
SVM (RBF)	57.15%
Random Forest	58.21%

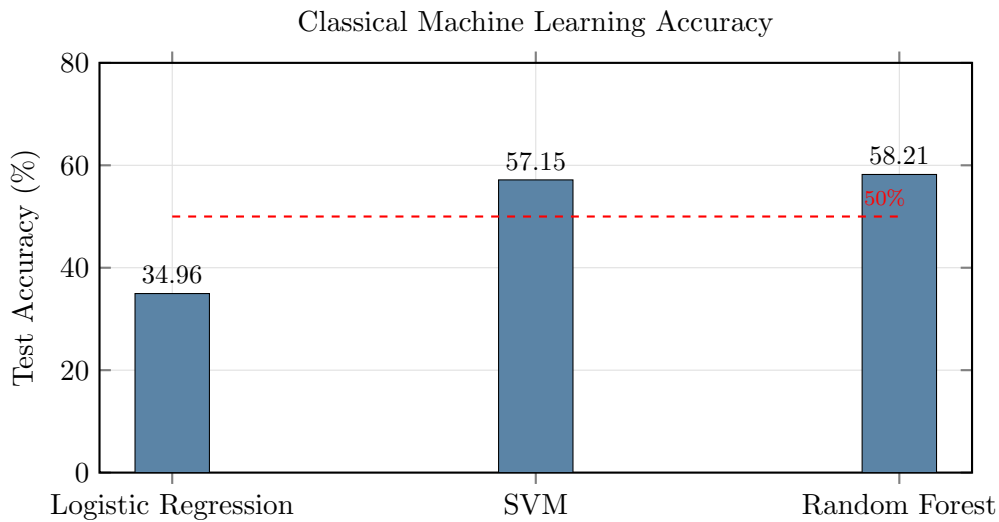


Figure 3: Random Forest achieved the best result among classical models.

The classical models were useful as baselines, but their performance was limited. Logistic Regression performed poorly because the problem is highly nonlinear. SVM and Random Forest performed better, but they still lost spatial information because the images were flattened.

6. Transfer Learning with EfficientNetB0

6.1 Reason for Choosing EfficientNetB0

EfficientNetB0 was selected because it is pretrained on ImageNet and can reuse strong visual features such as edges, textures, shapes, and object parts. This is more suitable for image classification than flattened pixel vectors.

The related papers also support this choice. WasteNet used transfer learning for smart-bin classification [1], while the CNN paper showed that convolutional architectures can learn visual waste patterns better than manual classification [2]. Our approach combines these ideas by using a pretrained CNN backbone and applying it to a 10-class garbage dataset.

6.2 Data Loading

EfficientNetB0 uses images of size 224×224 .

```

IMG_SIZE = 224
BATCH_SIZE = 16

def make_dataset(path, shuffle=False):
    ds = tf.keras.utils.image_dataset_from_directory(
        path,
        image_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        shuffle=shuffle,
        seed=42,
        label_mode="int"
    )
    return ds.prefetch(tf.data.AUTOTUNE)

```

The batch shape was:

(16, 224, 224, 3)

6.3 Data Augmentation

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.15, fill_mode="reflect"),
    tf.keras.layers.RandomZoom(0.15, fill_mode="reflect"),
    tf.keras.layers.RandomTranslation(0.1, 0.1, fill_mode="reflect"),
    tf.keras.layers.RandomContrast(0.15),
    tf.keras.layers.RandomBrightness(0.15, value_range=(0.0, 255.0)),
], name="augmentation")
```

Augmentation was used to improve generalization. It helps the model learn objects under different rotations, positions, brightness levels, and contrast conditions.

7. EfficientNetB0 Architecture

The following code builds the final EfficientNetB0 transfer learning architecture.

```
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, regularizers

base = EfficientNetB0(
    include_top=False,
    weights="imagenet",
    input_shape=(224, 224, 3)
)

base.trainable = False

inp = tf.keras.Input(shape=(224, 224, 3))
x = data_augmentation(inp)
x = layers.Lambda(preprocess_input)(x)
x = base(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dense(256, activation="relu",
                  kernel_regularizer=regularizers.l2(1e-4))(x)
x = layers.Dropout(0.4)(x)
x = layers.Dense(128, activation="relu",
                  kernel_regularizer=regularizers.l2(1e-4))(x)
x = layers.Dropout(0.3)(x)
out = layers.Dense(10, activation="softmax")(x)

model = tf.keras.Model(inp, out)
```

7.1 Parameter Summary

Parameter Type	Number
Total parameters	4,416,813
Trainable parameters	364,682
Non-trainable parameters	4,052,131

Only the custom classification head was trainable in the first phase. The EfficientNetB0 base was frozen.

8. Training

8.1 Phase 1: Feature Extraction

```
model.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),  
    metrics=["accuracy"]  
)  
  
callbacks = [  
    EarlyStopping(  
        monitor="val_accuracy",  
        patience=5,  
        restore_best_weights=True  
    ),  
    ReduceLROnPlateau(  
        monitor="val_loss",  
        factor=0.5,  
        patience=3  
    ),  
    ModelCheckpoint(  
        "best_phase1.keras",  
        monitor="val_accuracy",  
        save_best_only=True  
    )  
]  
  
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=20,  
    callbacks=callbacks  
)
```

The best validation accuracy was:

94.13%

Training stopped early at epoch 18 and restored the best model from epoch 13.

8.2 Training Accuracy Curve

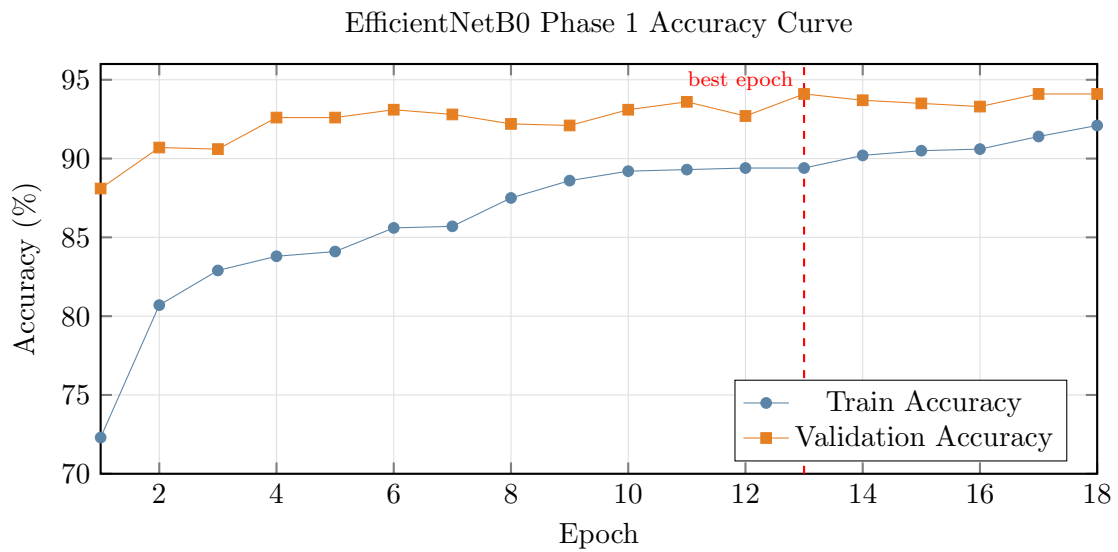


Figure 4: Validation accuracy reached 94.13%, showing that transfer learning worked well.

8.3 Training Loss Curve

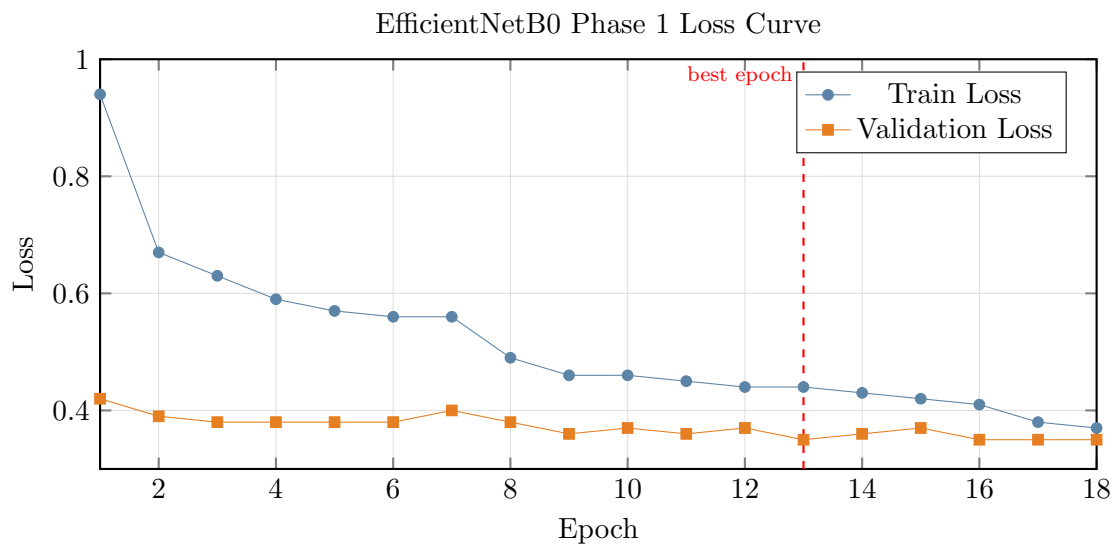


Figure 5: Training loss decreased steadily, while validation loss remained stable.

The accuracy and loss curves show that the model learned effectively. Validation accuracy stayed high and validation loss did not increase sharply, so there was no strong overfitting.

9. Final Evaluation

The best EfficientNetB0 model was evaluated on the test set.

```
best_model = tf.keras.models.load_model(
    "best_phase1.keras",
    custom_objects={"preprocess_input": preprocess_input}
)

test_loss, test_acc = best_model.evaluate(test_ds, verbose=0)
```

Metric	Value
Test Accuracy	92.68%
Test Loss	0.3500
Correct Predictions	1140 / 1230
Wrong Predictions	90 / 1230
Error Rate	7.3%

The final EfficientNetB0 model exceeded the target accuracy range of 85–90%.

10. Per-Class Metrics

Class	Precision	Recall	F1-score	Support
battery	0.97	0.99	0.98	76
biological	0.99	0.97	0.98	70
cardboard	0.92	0.94	0.93	142
clothes	0.98	0.98	0.98	190
glass	0.93	0.89	0.91	174
metal	0.78	0.92	0.85	93
paper	0.94	0.87	0.91	134
plastic	0.91	0.88	0.89	160
shoes	0.97	0.99	0.98	145
trash	0.77	0.78	0.77	46
Accuracy	0.93			
Macro Avg	0.92	0.92	0.92	1230
Weighted Avg	0.93	0.93	0.93	1230

10.1 F1-score by Class

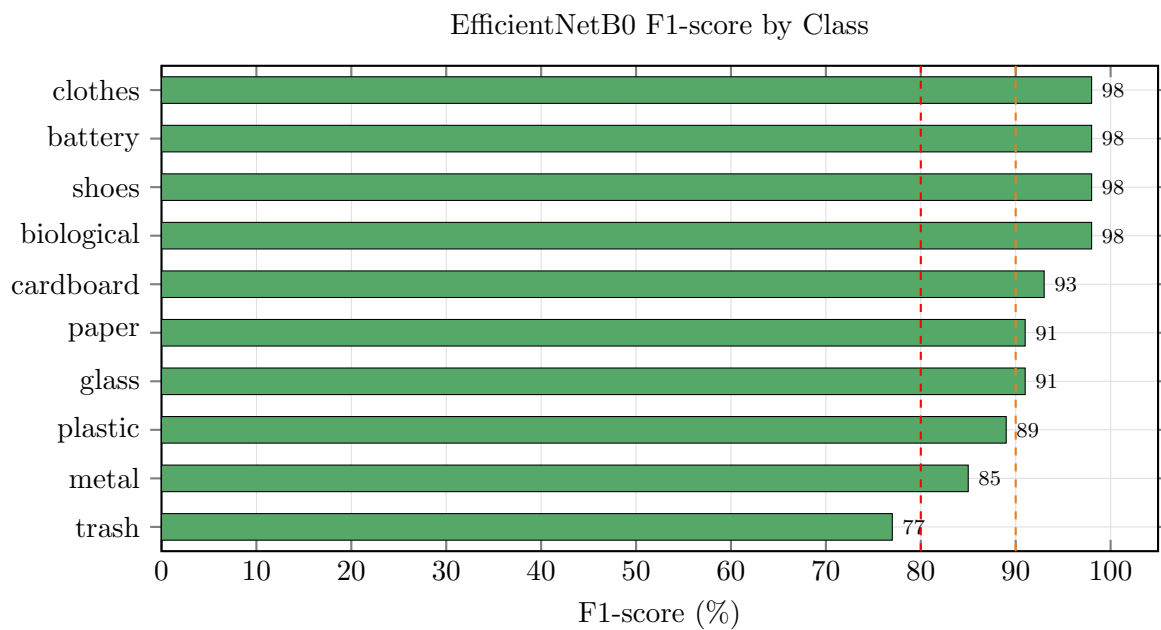


Figure 6: Trash and metal had the lowest F1-scores, while clothes, battery, biological, and shoes performed best.

10.2 Precision vs Recall

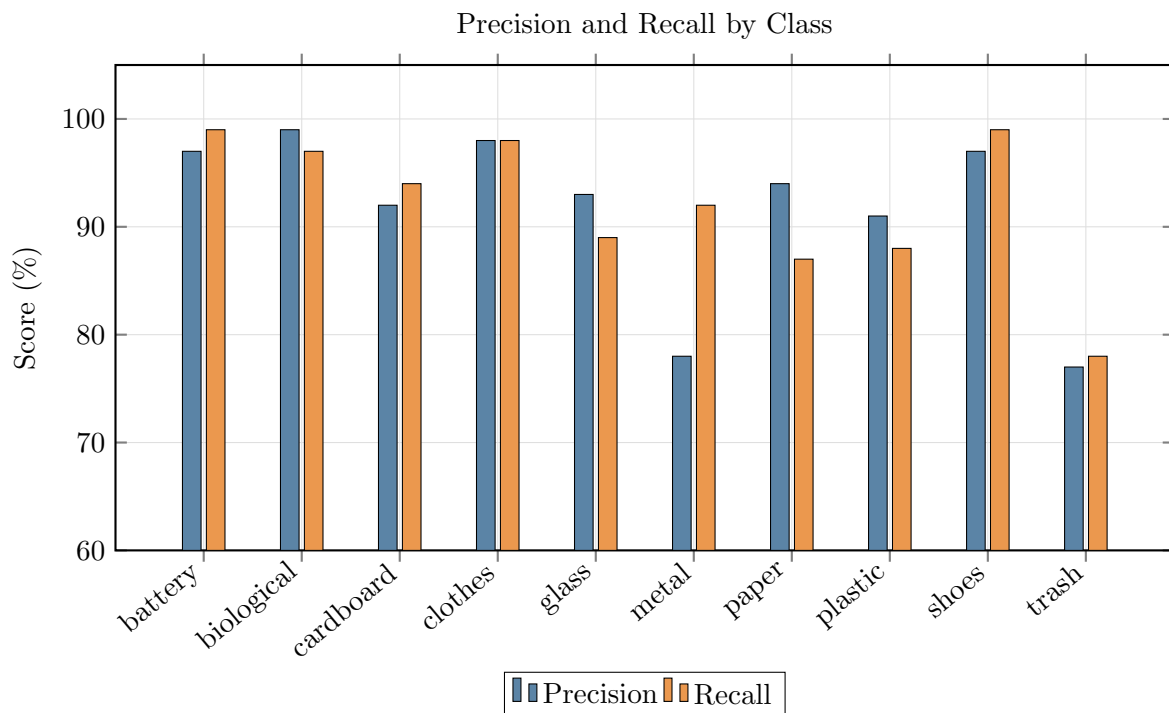


Figure 7: Metal has high recall but lower precision, meaning some other classes were predicted as metal.

11. Confusion Matrix and Error Analysis

The confusion matrix below shows how the final EfficientNetB0 model predicted each class on the test set. The absolute matrix shows the number of images, while the normalized matrix shows the

same results as proportions within each true class.

11.1 Absolute Confusion Matrix

True / Pred	bat	bio	card	cloth	glass	metal	paper	plast	shoes	trash
battery	75	0	0	0	0	1	0	0	0	0
biological	0	68	0	0	1	0	0	1	0	0
cardboard	0	0	133	0	2	1	3	0	2	1
clothes	0	0	0	187	0	0	1	1	1	0
glass	0	1	0	0	155	9	0	8	1	0
metal	2	0	1	0	2	86	0	1	0	1
paper	0	0	6	0	1	4	117	2	0	4
plastic	0	0	3	2	5	5	1	140	0	4
shoes	0	0	0	1	0	0	0	0	143	1
trash	0	0	2	0	1	4	2	1	0	36

11.2 Normalized Confusion Matrix

True / Pred	bat	bio	card	cloth	glass	metal	paper	plast	shoes	trash
battery	0.99	0.00	0.00	0.00	0.00	0.01	0.00	0.00	0.00	0.00
biological	0.00	0.97	0.00	0.00	0.01	0.00	0.00	0.01	0.00	0.00
cardboard	0.00	0.00	0.94	0.00	0.01	0.01	0.02	0.00	0.01	0.01
clothes	0.00	0.00	0.00	0.98	0.00	0.00	0.01	0.01	0.01	0.00
glass	0.00	0.01	0.00	0.00	0.89	0.05	0.00	0.05	0.01	0.00
metal	0.02	0.00	0.01	0.00	0.02	0.92	0.00	0.01	0.00	0.01
paper	0.00	0.00	0.04	0.00	0.01	0.03	0.87	0.01	0.00	0.03
plastic	0.00	0.00	0.02	0.01	0.03	0.03	0.01	0.88	0.00	0.03
shoes	0.00	0.00	0.00	0.01	0.00	0.00	0.00	0.00	0.99	0.01
trash	0.00	0.00	0.04	0.00	0.02	0.09	0.04	0.02	0.00	0.78

The diagonal values are high, which means that the model correctly classified most test images. The strongest classes were *battery*, *biological*, *clothes*, and *shoes*. The weakest class was *trash*, with only 78% normalized recall.

Most errors occurred between visually similar materials. For example, some *glass* images were predicted as *metal* or *plastic*. Some *trash* images were confused with *metal*, *paper*, and *cardboard*. This is expected because trash is visually diverse and has the smallest number of test examples.

12. Final Model Comparison

Model	Type	Test Accuracy
Logistic Regression	Classical ML	34.96%
SVM (RBF)	Classical ML	57.15%
Random Forest	Classical ML	58.21%
EfficientNetB0	Transfer Learning	92.68%

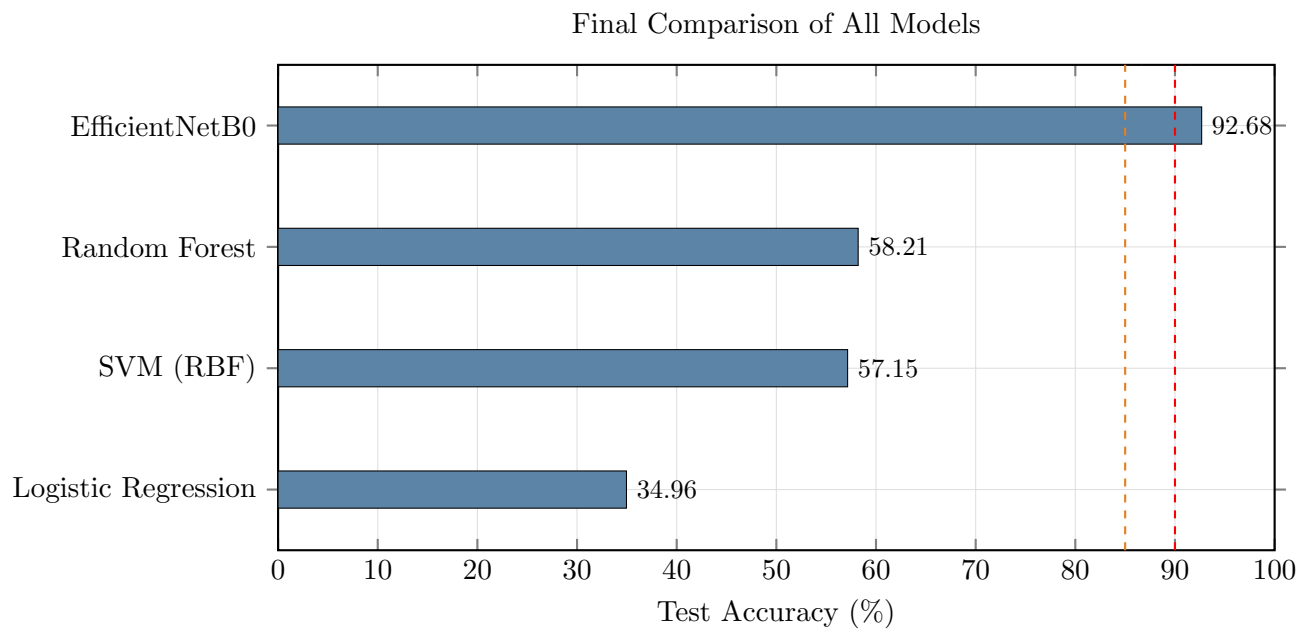


Figure 8: EfficientNetB0 achieved the highest accuracy among all models.

EfficientNetB0 improved accuracy by **34.47 percentage points** compared with Random Forest. This confirms that transfer learning is much more suitable for this image classification task than classical models using flattened pixels.

13. Final Model and Demo Description

The final model was saved and used in a local web application.

```
best_model.save("garbage_efficientnet_final.keras")

import json

with open("class_names.json", "w") as f:
    json.dump(class_names, f)
```

The application uses:

- the saved model file,
- class names JSON file,
- image upload interface,
- prediction endpoint.

The app allows users to upload a waste image and receive the predicted class.

14. Limitations

The project has several limitations:

- The dataset is imbalanced.
- The *trash* class has the weakest performance.

- The model cannot return an “unknown” class.
- Real-world images may contain multiple objects or dirty backgrounds.
- Production-level systems require more images from real environments.

15. Future Work

Future improvements include:

- adding more images for minority classes,
- trying EfficientNetB2 or EfficientNetB3,
- applying fine-tuning to lower EfficientNet layers,
- adding confidence thresholding,
- using macro F1-score as the main metric,
- deploying the app online.

16. Analysis and Conclusions

This project successfully developed a smart garbage classification system. Classical machine learning models were trained as baselines, but their performance was limited because they used flattened image features.

The final EfficientNetB0 transfer learning model achieved **92.68%** test accuracy and clearly outperformed all baseline models. The model performed especially well on battery, biological, clothes, and shoes. The weakest class was trash, mainly because of limited data and visual diversity.

Compared with related work, our project uses more classes than the common 6-class TrashNet setup and provides a clear comparison between classical machine learning and transfer learning. Overall, EfficientNetB0 was the best model for this task and satisfies the project requirements for dataset preparation, exploratory analysis, multiple model experiments, performance comparison, error analysis, and interactive demonstration.

References

- [1] G. White, C. Cabrera, A. Palade, F. Li, and S. Clarke, “WasteNet: Waste Classification at the Edge for Smart Bins,” arXiv:2006.05873, 2020.
- [2] A. M. Al-Mamun, R. Hossain, M. M. M. Sharmin, E. Kabir, and M. A. Iqbal, “Garbage classification using convolutional neural networks (CNNs),” *Material Science & Engineering International Journal*, vol. 7, no. 3, pp. 140–144, 2023.

Course: CSS 324 — Introduction to Machine Learning

University: SDU University

Semester: Spring 2026